SERENDIPITY

SOLUTION ENGINEER RESPONSE · PERRICONE FARMS

# HUBSPOT REPORT RECREATION

A practical, proactively-scoped response to the Perricone Farms consolidated sales report — mapping plan, data model, dashboard strategy, and integration architecture.


PREPARED BY

**Gab Narvadez**

Solution Engineer


FOR

**Lighthouse — Megan & AS**

Client: Perricone Farms

DATE

**April 2026**

v1.0

CONTEXT

# 0 · WHAT I READ BEFORE ANSWERING

I went through Megan's walkthrough and the two data files. Before I propose anything, here's what the report actually is, in plain language, so we're aligned on the problem we're solving:

- It's a **Consolidated Sales Report** with five lenses on the same underlying data: **By Customer, By Sector, By Plant, By Sales Rep, By Month**.
- The core metrics are **Gallons Sold**, **Gross Sales ($)**, **vs Budget**, **vs Prior Year**, **Mix %**, **ASP** (avg selling price), plus **AR aging trend** per plant.
- There are two sources because there are two physical plants:
  - **Batchmaster → Beaumont plant** (API available)
  - **Vero → Vero Beach plant** (no API — manual export)
- Some customers appear in **both** plants (e.g., "COMBINED TRADER JOE'S", "COMBINED COSTCO"). The Excel manually rolls these up.
- The report subtracts **intercompany** transactions from consolidated totals.
- Budget is stored per customer, per month, at the gallon and dollar level.
- Sector ("Foodservice", "Retail", "Private Label", "Bulk", "PL Foodservice") is a manual mapping table on Sheet1 — it is **not** in either ERP.

> THE FRAMING
>
> The problem is not "get data into HubSpot." It's **recreate a BI-style rollup inside a CRM** that wasn't built to be a finance tool. That framing drives every decision below.

PROACTIVE DISCOVERY

# 1 · WHAT I'D ASK THE CLIENT BEFORE TOUCHING HUBSPOT

This is the "proactive problem-solving" piece. Before I build anything I want answers to these — they determine the scope dramatically:

## ABOUT THE DATA

1. Can we get a **sample Vero export file** the way their team actually receives it? The workbook I have is a combined one; I want to see Vero's raw format.
2. What's the **cadence** of the Vero export? Daily, weekly, at month-close? Who does it?

3. Is there a **customer master** anywhere that already reconciles customers across both ERPs, or is the matching logic only in the Excel?
4. How are **intercompany** transactions flagged in the source data? A customer # range? A product code? A plant pairing? Right now the Excel has a hard-coded "Less intercompany" row.
5. How are **new customers** set up today? Is a sector assigned at the time of onboarding, or is it retro-filled in the Excel?
6. **Budgets** — are they set once a year, or revised? Who owns the budget file?
7. **AR data** — where does that come from? The workbook has BM and VB AR aging trends separately; is that also a Batchmaster/Vero export, or from an accounting system (NetSuite, QuickBooks)?

## ABOUT THE HUBSPOT SIDE

8. What HubSpot tier are they on? **Custom objects require Enterprise** (Sales Hub Enterprise or Operations Hub Enterprise). If they're on Pro, the solution looks different — we'd lean heavier on CRM pipelines plus external BI.
9. Who consumes this report today, and in what format — PDF, dashboard, deck? That changes whether we need the **exact visual layout** replicated or just the insights.
10. Is HubSpot replacing the Excel, or **supplementing** it? A lot of clients ask for replacement but actually want "the same thing, plus filters and sharing." Very different scope.

# 2 · STEPS TO ACCURATELY MAP THE DATA INTO HUBSPOT

The Excel has 60+ tabs, but the raw data underneath is surprisingly simple. Every row from both ERPs is the same shape:

```
Date (month) | Customer # | Customer Name | Store # | Store Name |
Flavor Code | Flavor Desc | Product Code | Product Desc |
Gallons | Qty | Rate | Amount
```

That's one **sales line item**. Every calculation in the workbook — the consolidated report, the sector rollup, the rep dashboard, everything — is a pivot of this table plus a budget table plus a customer-to-sector lookup.

## MY 7-STEP MAPPING PLAN

### 01 BUILD A CANONICAL DATA DICTIONARY

One document that lists every source field from Batchmaster and Vero, the HubSpot destination property, the data type, transformation rules, and edge cases. This is the contract we build against. I'd review it with the AS and the client's finance lead before touching the portal.

## 02 DESIGN THE HUBSPOT OBJECT MODEL

The most important decision, and where I'd push back on any prior suggestion to use **Deals** for sales transactions. Deals imply pipeline stages and sales cycles — these are closed shipments. Naming a deal "gallons sold" would be exactly the kind of surface-level decision that confuses the client down the road and makes every dashboard harder to build.

**Proposed model:**

| HUBSPOT OBJECT | WHAT IT REPRESENTS | WHY |
|---|---|---|
| Company (native) | Customer account | One record per customer; holds the sector, sales rep, plant(s) |
| Contact (native) | Customer-side buyer | Standard CRM use — not load-bearing for this report |
| Product (native) | SKU / flavor catalog | The "what we sell" list. Price, cost, description. ~40 records, rarely changes |
| Custom Object: Sales Transaction | One invoice line, per month, per product, per customer | The "what was sold, when, to whom." The atomic unit of the report. Associated to Company AND Product |
| Custom Object: Budget Line | One budget entry per customer per month (gallons + $) | Enables "vs Budget" without hard-coding |
| Custom Object: AR Snapshot | Monthly AR aging per customer per plant | Lets us replicate the AR trend tabs |

**ANTICIPATED FOLLOW-UP — "IF DEALS ARE WRONG, WHY NOT JUST USE PRODUCTS?"**

Because Products and Sales Transactions do different jobs. A **Product** is a catalog entry — one record per SKU, static, defines *what we sell*. A **Sales Transaction** is an event — one record per customer × product × month, dynamic, defines *what was sold, when, to whom*. You can't store transactional data on a Product without polluting the catalog. And HubSpot's native "a sale happened" object — **Line Items** — only exists as a child of a Deal or Quote, which forces us back into the Deal anti-pattern. Sales Transaction as a standalone custom object is the clean escape. **We use Products and Sales Transaction together**, not one instead of the other.

## 03 DEFINE THE PROPERTY SET FOR EACH OBJECT

### COMPANY (ADDITIONS TO NATIVE):

- `ERP Customer ID — Batchmaster` (unique, text)
- `ERP Customer ID — Vero` (unique, text)
- `Active in plant` (multi-select: Beaumont, Vero Beach)
- `Sector` (single-select: Foodservice, Retail, Private Label, Bulk, PL Foodservice)
- `Sales Rep` (HubSpot user or single-select, matches the rep tabs: Daryl, Julie, Israel, Jaime, Bryan, Olga)
- `Is Intercompany` (boolean)

### SALES TRANSACTION (CUSTOM OBJECT):

- `Transaction ID` (unique — hash of source + customer + product + month + line#)
- `Transaction Month` (date, always 1st of month for consistency)
- `Source ERP` (Batchmaster / Vero)
- `Plant` (Beaumont / Vero Beach)
- `Gallons`, `Quantity`, `Rate`, `Amount`
- Associations: Company (customer), Product

### BUDGET LINE (CUSTOM OBJECT):

- `Budget Period` (month, year)
- `Budget Gallons`, `Budget Sales $`
- Association: Company

## 04 RECONCILE CUSTOMERS ACROSS THE TWO ERPS

This is the trap. "TRADER JOE'S" in Batchmaster and "TRADER JOE'S" in Vero are the same company but different IDs. I'd:

- Build a **match table** using exact-name + fuzzy-match + manual review
- Land it as two separate ERP customer ID fields on a single Company record
- Flag anything unmatched for the client's finance team to resolve before go-live

## 05 HANDLE THE SECTOR AND REP MAPPINGS

Today these live in Excel lookup tables. In HubSpot they become Company properties. I'd load them once from Sheet1 and the rep tabs, then maintain them via a **single workflow** so reps and finance don't both update in different places.

## 06 NORMALIZE THE DATE FIELD

The raw data stores month as a string ("January", "February"). Every "MTD" and "YTD" calculation in HubSpot depends on having a real date. I'd standardize to the 1st of the month (or a dated transaction-effective-date if the ERPs expose one — worth asking).

## 07 PILOT IMPORT AND RECONCILIATION BEFORE CUTOVER

- Load **one full month** (say, January 2024) of Batchmaster data
- Regenerate every number from that month's tab in the Excel, inside HubSpot
- Tie the totals out **to the penny** against the client's Excel
- Only after the pilot passes do we extend to full historical load (2023 + 2024 YTD) and wire up the API

> **NON-NEGOTIABLE**
>
> This step is the one I'd never cut. The client's trust in the new system lives or dies here.

# 3 · HOW I'D RECREATE THE REPORT INSIDE HUBSPOT

HubSpot is a CRM, not a BI tool. I want to set the expectation with the client upfront: **we will replicate the numbers and the filters, not the exact visual layout** of a 60-tab Excel. That's a feature, not a compromise — the Excel has ten versions of the same data because that's how Excel forces you to slice things. HubSpot can give them one dashboard with a month selector.

## THE LAYERED APPROACH

### LAYER 1 — NATIVE HUBSPOT CUSTOM REPORTS + DASHBOARDS

Once the data model is live, these are trivial to build and become real-time the moment the Batchmaster API is wired up:

- **Top Customers dashboard** — Gallons and Sales, MTD/YTD, with change vs PY, grouped by Sector
- **Sector performance dashboard** — totals by sector, with filters for plant
- **Plant comparison** — Beaumont vs Vero Beach, MTD and YTD
- **Sales Rep dashboard** — one per rep (Daryl, Julie, etc.), auto-filtered by rep ownership
- **AR Aging trend** — line chart per plant from the AR Snapshot object

Each of these uses a **month filter** at the top of the dashboard — that replaces the "< Select Here" dropdown in the current Excel.

### LAYER 2 — CALCULATED PROPERTIES FOR DERIVED METRICS

The Excel has a lot of "vs Budget %", "vs PY %", and "Mix %" columns. In HubSpot:

- `Variance vs Budget ($)` = Amount – Budget Amount (calculated property on Company or rollup)
- `Variance vs Budget (%)` = (Amount – Budget) ÷ Budget
- `YoY %` = (Current Year Amount – Prior Year Amount) ÷ Prior Year Amount
- `Mix %` = Customer Amount ÷ Total Amount *(this one is the trickiest — may need a scheduled workflow)*

### LAYER 3 — WORKFLOWS FOR ROLLUPS

HubSpot native reports can aggregate Sales Transactions up to Company, but for MTD/YTD we'll use a scheduled workflow that refreshes `Company.YTD Gallons`, `Company.YTD Sales`, etc. nightly. This keeps dashboards fast.

### LAYER 4 — THE EXEC-SUMMARY LAYOUT

The client has a finance-style PDF layout (Exec Summary By Plant, etc.). HubSpot dashboards won't produce that exact format. Two options I'd present:

> **OPTION A — RECOMMENDED: DASHBOARD-NATIVE**
>
> Rebuild the exec summary as a set of HubSpot dashboard widgets. Finance can share a link, filter by month, export to PDF. **Pros:** single source of truth, real-time, no extra tooling. **Cons:** doesn't match the exact visual layout.

> **OPTION B — BI LAYER ON TOP**
>
> Connect a tool like **Hex, Databox, Sigma, or Google Sheets via HubSpot's native connector** to pull the data out for the exact PDF layout. **Pros:** client keeps their preferred format. **Cons:** another system to maintain, and the numbers are now in two places.

I'd take this decision back to the client rather than assume. That's a conversation the AS and I run together.

## WHY THIS WORKS

Every number in the current Excel can be re-derived from: a Sales Transaction table + a Budget table + a Customer-to-Sector lookup. We have all three. HubSpot's custom reports can do sums, variances, groupings, and time-series on custom objects. The only thing we're giving up is Excel's spatial flexibility, which is a cost-benefit the client has already implicitly accepted by asking for this in the first place.

# 4 · THE INTEGRATION ARCHITECTURE

Megan asked about the eventual API migration and the Vero drop-in cadence. My high-level view:

## BATCHMASTER (API)

- Build a middleware service (simplest: a small scheduled job — Python/Node, or a no-code option like Make/Workato if the client prefers) that polls Batchmaster on a **daily** cadence.
- Transform to HubSpot schema, upsert via HubSpot CRM API using `Transaction ID` as the unique key (idempotent — re-running a day is safe).
- Financial data rarely needs sub-hour latency. **Daily is the right cadence unless the client tells us otherwise.** I'd push back on "real-time" as over-engineering.

## VERO (NO API)

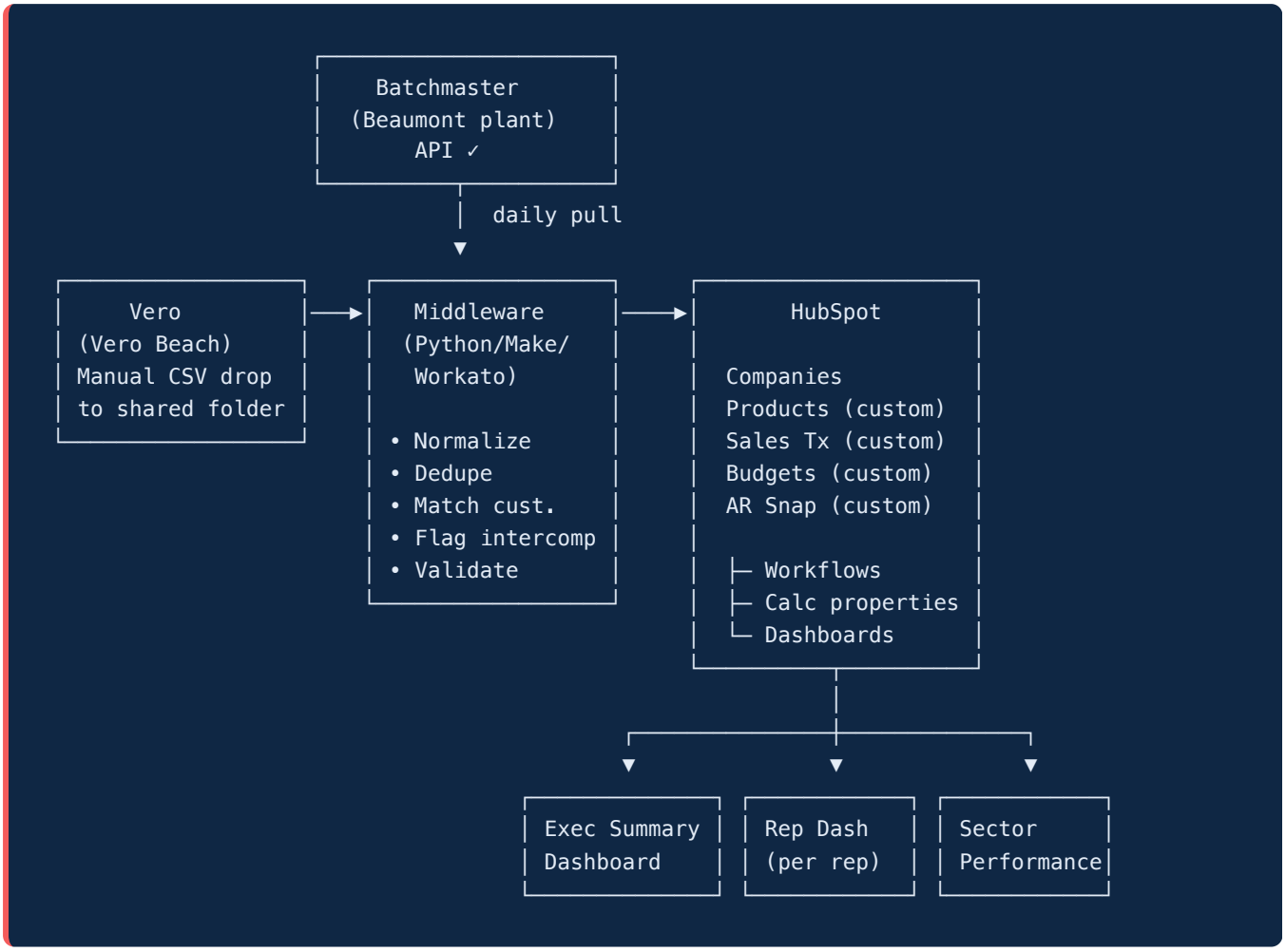
- Set up a shared folder (SharePoint / Google Drive) where their team drops the export.
- A watcher script validates schema, transforms, upserts — same pipeline as Batchmaster, different ingestion.
- **Cadence should match their close cycle.** If Vero closes monthly, monthly import is fine. If they need mid-month visibility, weekly.
- Put **validation guardrails** on the import — flag row-count deltas, schema drift, duplicate keys — before the data hits HubSpot. The worst failure mode is a silently corrupt month.

## THINGS I'D FLAG TO THE CLIENT NOW

1. If a customer gets renamed or re-IDed in either ERP, we need a renaming protocol. Otherwise we'll create duplicate Company records.
2. If the ERP doesn't expose a reliable "last modified" timestamp, we have to full-refresh the current month each run. Worth confirming Batchmaster supports incremental.
3. Intercompany logic should live in the middleware, not the dashboard. Tag it on ingest.
4. Is there an upstream accounting system (NetSuite, QB) that already consolidates BM + Vero? If yes, it might be the better source than pulling each ERP separately. Don't rebuild consolidation logic that's already authoritative elsewhere.

ARCHITECTURE

## 5 · SOLUTION ARCHITECTURE




MEETING PREP

## 6 · WHAT I'D BRING TO THE CLIENT MEETING

Per the SE rubric ("go beyond suggestions"), I'd walk in with:

1. This document as the written record.
2. A **3-slide deck** for the AS to present: problem → proposed architecture → phased rollout.
3. A **HubSpot sandbox portal** with the custom objects already created and one month of pilot data loaded, so the client can click through a working dashboard instead of imagining one.
4. A **phased rollout plan**: Week 1–2 data model + pilot import; Week 3–4 dashboards + reconciliation; Week 5–6 Batchmaster API live; Week 7+ Vero cadence + AR integration.

**WHY THE SANDBOX**

The sandbox is the part that makes the meeting feel real. Excel mockups of dashboards are a red flag — if I can't stand up a working example in a sandbox in a week, the solution probably has a gap I haven't found yet.

# 7 · DEEP-LEVEL NAMING

The rubric called out not naming things like "gallons sold." Here's how I'd name the key pieces, and why:

| THING | BAD NAME | GOOD NAME | WHY |
|---|---|---|---|
| The atomic sales record | "Gallons Sold" deal | **Sales Transaction** | It's a transaction, not a deal. "Gallons" is one metric on it, not its identity. |
| The budget record | "Budget Deal" | **Budget Line** | Matches finance's mental model. Parallel to "Sales Transaction." |
| The Batchmaster → HubSpot pipe | "BM Sync" | **Beaumont Plant Ingestion** | Client thinks in plants, not ERP brand names. Future-proofs if they switch ERPs. |
| The sector field | "Type" | **Channel / Sector** | "Type" is meaningless. Channel is the word their sales team actually uses. |
| Custom dashboard | "Sales Report" | **Consolidated Sales Performance — [Month]** | Matches the Excel title they already know. Reduces training friction. |

Small, but these are the cues the AS will repeat to the client for the next year. Worth getting right on day one.